



Universidade do Minho

Licenciatura em Engenharia Informática

Fase 2

Projeto LI3

PL6- Grupo 63

Pedro Afonso Quesado Pacheco Pereira (A107332)

José Dinis Guimarães Machado (A106926)

Octávio Pita Henriques (A10427)

1.Introdução

A segunda fase do projeto de Laboratórios de Informática III (2025/2026) representa a consolidação e o amadurecimento do sistema de gestão de dados iniciado na etapa anterior. O foco central desta fase residiu na expansão da robustez do sistema para lidar com um volume de dados superior e na implementação de outro modo de execução, programa interativo.

Com a introdução de um executável interativo, o sistema passou a oferecer uma interface terminal que permite ao utilizador carregar *datasets* e consultar informações de forma dinâmica, garantindo a validação contínua dos inputs introduzidos. Paralelamente, a conclusão das interrogações obrigatórias — incluindo as novas **Queries 4, 5 e 6**, bem como a atualização da **Query 1** para incluir contagens de passageiros — permitiu um leque analítico mais profundo sobre a interação entre voos, passageiros e aeroportos.

Tecnicamente, o desafio desta fase centrou-se na aplicação rigorosa dos conceitos de **encapsulamento** e **modularidade**, utilizando uma estrutura de gestão centralizada `manager_c` para orquestrar os diversos catálogos de entidades. Para garantir a escalabilidade exigida por interrogações complexas, foram adotadas estruturas de dados avançadas e técnicas de pré-agregação estatística durante o *parsing* dos ficheiros CSV. Por fim, a introdução de um programa de testes dedicado permitiu aferir a eficiência temporal e a integridade da gestão de memória, assegurando, através do uso do **Valgrind**, uma solução otimizada e livre de *memory leaks*.

2.Nova arquitetura e estruturas

2.1 Alterações iniciais

Durante a evolução do projeto para a Fase 2, foram realizados ajustes estruturais e refinamentos focados na escalabilidade do sistema para processar *datasets* de maior volume. Simplificaram-se as entidades principais, removendo campos redundantes para otimizar o consumo de memória.

A principal mudança estratégica consistiu na introdução de uma técnica de pré-agregação de dados durante a fase de carregamento (*parsing*). Ao contrário de uma abordagem ingênua que processaria todos os dados no momento da consulta, o nosso sistema calcula as estatísticas necessárias para as novas interrogações à medida que os ficheiros CSV são lidos. Esta revisão eliminou a necessidade de módulos de estatísticas complexos e isolados, integrando-os diretamente no gestor de catálogos e nos parsers, o que resultou num sistema mais ágil e eficiente.

2.2 Arquitetura

A arquitetura do projeto manteve-se fiel aos princípios de modularidade, separando claramente o Processamento de Entrada (*Parsers*), a Lógica de Negócios (Catálogos e Entidades) e a Geração de Resultados (*Interpreter e Output*).

O sistema é agora orquestrado por um módulo gestor central, o `manager_c`, que encapsula todos os catálogos de entidades e as novas estruturas auxiliares de estatísticas. O encapsulamento foi rigorosamente aplicado, mantendo a definição das structs privadas nos ficheiros `.c` e expondo apenas as interfaces necessárias através dos cabeçalhos `.h`. Esta estrutura permitiu a incorporação fluida do modo interativo sem comprometer a estabilidade do núcleo de processamento já estabelecido na fase anterior.

2.3 Arquitetura

Query 1 (Atualizada): "Listar o resumo de um aeroporto, incluindo o número de passageiros que aterraram e partiram."

A Query 1 foi reformulada nesta fase para incluir métricas estatísticas de fluxo de passageiros. Para garantir que a resposta à interrogação seja instantânea, a estrutura da entidade `Airport` foi expandida com os campos `total_arrivals` e `total_departures`. Estes contadores são atualizados dinamicamente durante o carregamento das reservas, onde o sistema identifica os voos associados e incrementa os valores nos aeroportos de origem e destino,

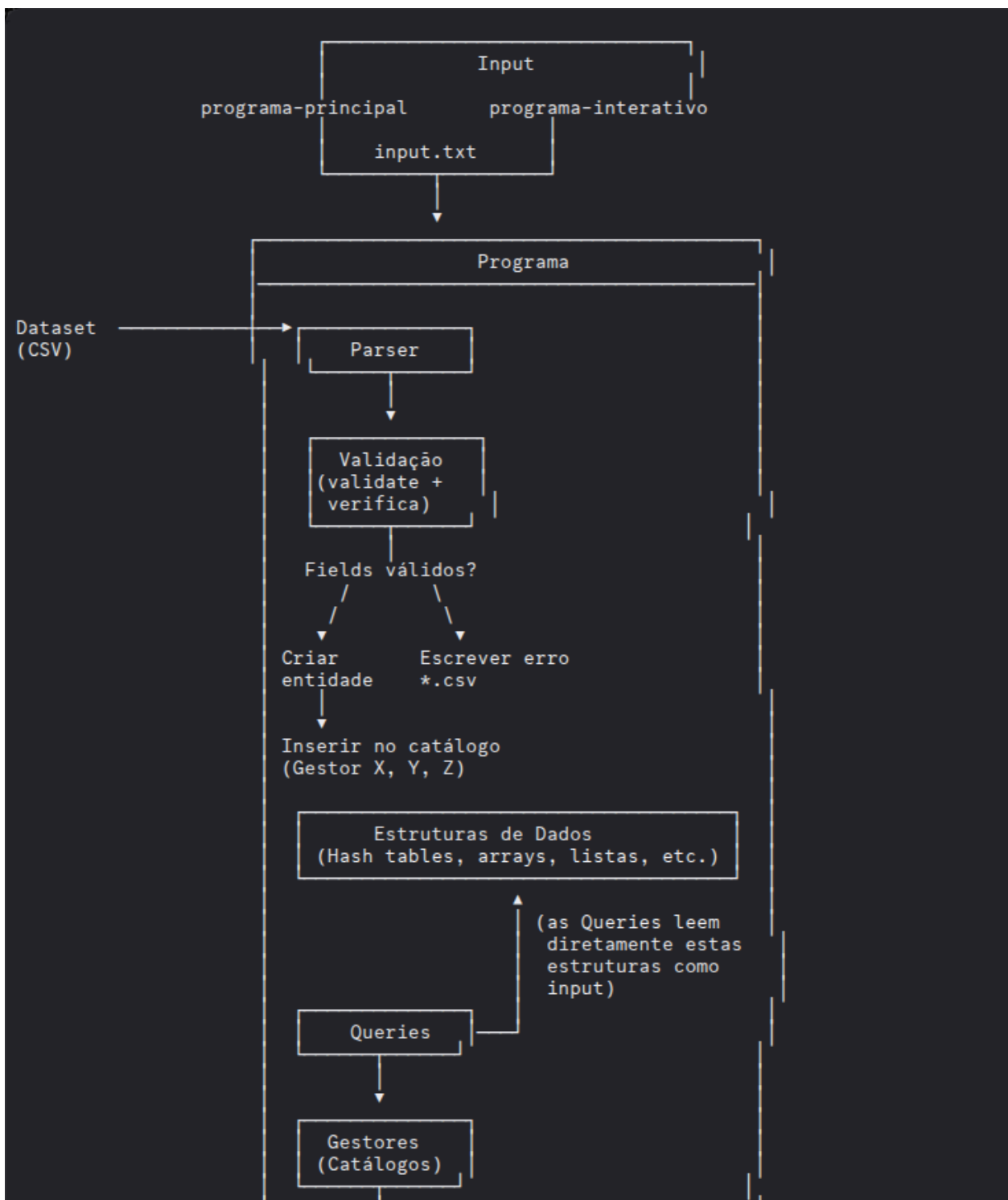
desde que o estado do voo não seja "Cancelled". Desta forma, a query executa em tempo constante $O(1)$, consultando diretamente os campos encapsulados na entidade após a validação do identificador.

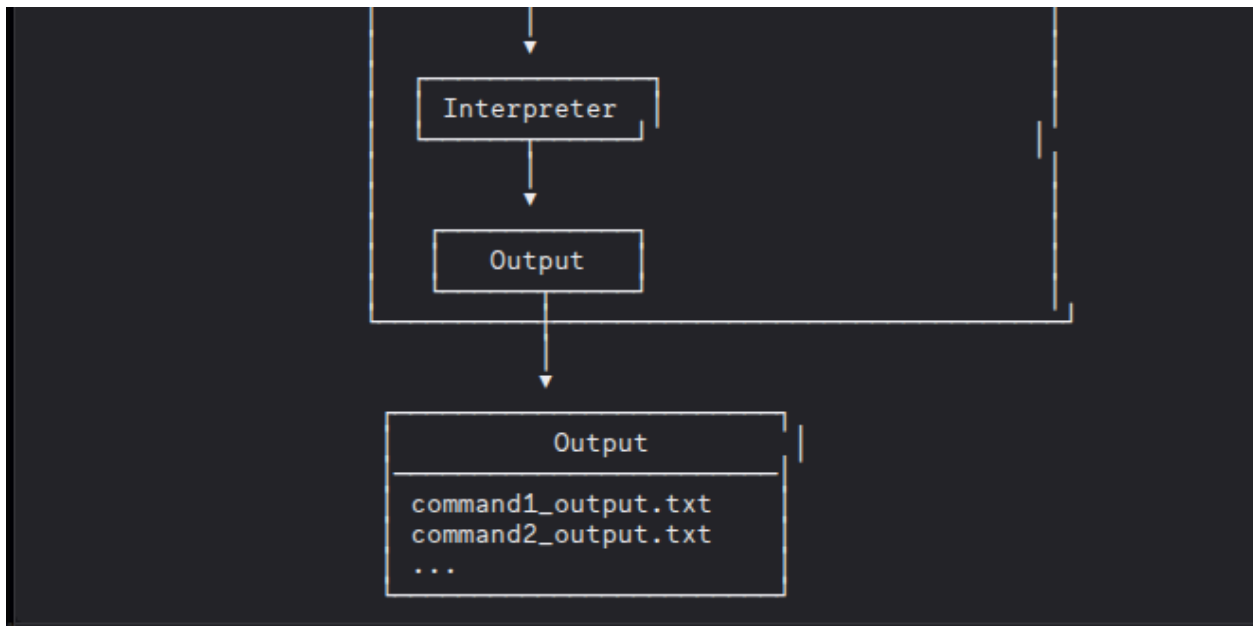
Query 5: "Top N companhias aéreas com maior tempo de atraso médio por voo."

A Query 5 foca-se na análise de performance das companhias aéreas, contabilizando apenas voos com o estado "Delayed". A implementação utiliza uma tabela hash denominada `airline_stats`, gerida pelo, que armazena estruturas `airline_stats` contendo o acumulado de minutos de atraso e o número de voos para cada companhia. O cálculo do atraso (diferença entre a partida real e estimada) é realizado no momento do *parsing* dos voos, otimizando a fase de execução da query. No momento da interrogação, os dados são ordenados decrescentemente pelo atraso médio — com arredondamento rigoroso a 3 casas decimais — utilizando a função `g_list_sorts` com critérios de desempate alfabético pelo nome da companhia

Query 6: "Listar o aeroporto de destino mais comum para passageiros de uma determinada nacionalidade."

A Query 6 utiliza uma estrutura de dados avançada de tabelas hash aninhadas (`nationality_stats`) para mapear nacionalidades a sub-tabelas de destinos. Durante o processamento das reservas, o sistema cruza a nacionalidade do passageiro com o aeroporto de destino dos seus voos, atualizando os contadores na estrutura global. Quando invocada, a query acede diretamente à sub-tabela da nacionalidade solicitada e identifica o aeroporto com a maior contagem de chegadas. Em caso de empate na contagem, o código IATA do aeroporto é utilizado como critério de desempate lexicográfico, garantindo a precisão dos resultados sem a necessidade de percorrer exaustivamente o catálogo de passageiros durante a consulta.





3. Modo Interativo

O modo interativo, materializado no executável `programa_interativo`, foi concebido para permitir uma exploração dinâmica do *dataset* através de uma interface de linha de comando (CLI). Ao contrário do modo *batch*, esta componente foca-se na experiência do utilizador, permitindo a execução sucessiva de múltiplas interrogações sem a necessidade de recarregar as estruturas de dados.

3.1 Funcionamento e Implementação

A implementação do modo interativo no projeto segue as diretrizes técnicas estabelecidas para a segunda fase, focando-se nos seguintes aspetos:

- **Carregamento de Dados:** O programa foi desenhado para iniciar sem argumentos de linha de comando. Embora o enunciado sugira perguntar o

caminho ao utilizador, o sistema utiliza a flexibilidade permitida de assumir um caminho *default* (`DEFAULT_DATASET_PATH`) por conveniência, garantindo que o utilizador possa iniciar as consultas imediatamente após a execução.

Ciclo de Interrogação: Foi implementado um ciclo de leitura contínuo (`while loop`) que utiliza a função `fgets` para capturar as entradas do utilizador. Este ciclo permanece ativo até que o comando `sair` ou `exit` seja introduzido, momento em que o sistema procede à libertação total da memória alocada.

Parser de Comandos: Para o processamento das *queries*, o programa utiliza a função `strtok` para separar o identificador da query dos seus respetivos argumentos. Esta abordagem permite identificar rapidamente se o utilizador pretende executar, por exemplo, a Query 6 focada em nacionalidades ou a Query 2 sobre o ranking de aeronaves.

3.2 Robustez e Validação

Um dos pontos centrais no desenvolvimento desta interface foi o cumprimento da instrução de **validação de dados** para evitar comportamentos erráticos ou falhas críticas (*crashes*).

Tratamento de Inputs: O sistema verifica se a entrada está vazia antes de tentar processar qualquer token. Além disso, o uso de um `switch` na lógica de encaminhamento garante que apenas identificadores de queries suportados (Q1, Q2, Q3, Q5 e Q6) sejam executados.

Gestão de Memória: Para garantir a integridade do sistema durante utilizações prolongadas, cada resultado gerado por uma query é apresentado no terminal e, de seguida, a memória associada a essa string de output é imediatamente libertada através da função `free()`.

3.3 Interface final

```
diniss_1@Dinis:~/ano3/LI3/2526-663/trabalho-pratico$ ./programa-iterativo
=== MODO INTERATIVO ===
A carregar dataset de: '/home/diniss_1/ano3/LI3/dataset-fase-1/sem_errores'...
>>> Dados carregados com sucesso! <<<
Insira a query (ex: 6 Portugal) ou 'sair':
>>
```

A imagem demonstra o arranque do executável **programa-iterativo**. Como observado, o sistema inicia com o carregamento automático dos catálogos a partir de um caminho predefinido (conforme permitido por conveniência no enunciado). O utilizador recebe *feedback* imediato sobre o sucesso da operação ("Dados carregados com sucesso!"), seguindo-se o prompt de espera para comandos. Esta abordagem cumpre o requisito de iniciar sem a necessidade de argumentos externos.

```
>> 5 10
JetVenture Airlines;5389;53.060
Sunrise Networks;7855;52.962
Cloud Nine;5573;52.913
Keen Skyways;6560;52.797
Global Way;5802;52.792
Liberty Plane;5716;52.772
BlueDawn Aviation;4713;52.770
Rivet Transport;5820;52.768
SkyGate Airlines;5620;52.752
Moonlight Airways;7679;52.674
>>
```

Terminal

Neste snapshot, observa-se a execução da Query 5 com o argumento **10** (Top 10). O output demonstra a precisão do motor de cálculo: as companhias são listadas com o número de voos atrasados e a respetiva média de atraso. É importante destacar que o sistema respeita rigorosamente a regra de arredondamento a 3 casas decimais (ex: 53.060), conforme exigido pela especificação da Fase 2. Os resultados comprovam a eficiência da pré-agregação de dados feita durante o parsing inicial.

```
>> 6 France
LRU;3
```


A imagem ilustra a simplicidade e eficácia da Query 6 no modo interativo. Ao introduzir **6 France**, o sistema acede à estrutura de tabelas hash aninhadas para devolver instantaneamente o aeroporto de destino mais comum para passageiros franceses (**LRU**) e a respetiva contagem (**3**). Este resultado reflete a lógica de ignorar voos cancelados e aplicar desempates lexicográficos, garantindo a integridade da resposta.

4. Conclusão

O desenvolvimento desta segunda fase permitiu consolidar os conhecimentos de programação estruturada em C, com especial enfoque na criação de sistemas modulares e resilientes. A implementação do modo interativo e do programa de testes transformou o motor de queries numa aplicação completa, capaz de interagir com o utilizador e auto-validar a sua correção

Como aspetos mais positivos, destaca-se a eficácia das hashtables na otimização das queries estatísticas e a ausência total de *memory leaks*, confirmada pela análise rigorosa com o Valgrind. Estas conquistas demonstram um cuidado minucioso na gestão de recursos dinâmico

As principais dificuldades sentidas prenderam-se com a gestão de argumentos variáveis no modo interativo (como nomes de nacionalidades com espaços) e a sincronização das validações lógicas entre diferentes catálogos. Como melhoria futura, seria interessante expandir o programa para suportar uma interface gráfica mais avançada ou a persistência dos dados em ficheiros binários indexados para reduzir o tempo de carregamento inicial. Em suma, o projeto atinge os objetivos propostos pelo enunciado, resultando numa ferramenta robusta e escalável para a gestão de dados aeronáuticos .

